



"The best way to predict the future is to invent it."

— ALAN KAY, 1971

Meet Streamlit.

Today your Python stops being scripts that print to a terminal. It becomes an app people can click through — in a browser, no HTML, no CSS, no JavaScript. Just Python.

PROGRAM

Python Internship

LED BY

Zlaark

SESSION

4 of 24



— TODAY'S FRAME

The best way to predict the future is to *invent* it.

— ALAN KAY — *Xerox PARC, 1971*

Alan Kay invented the graphical user interface — the windows, icons, and mouse you've used your whole life. Today, you build your **first UI**. Not in HTML or JavaScript — in Python, with Streamlit. The future of your project starts with one line: `st.title("My App")`.



How we'll spend our 90 minutes today.

More keyboard than Day 3. Today is the day your Python becomes something you can see and click. Every concept gets typed live.

DURATION

90 min

00:00 - 00:10



Day 3 Recap & Q&A

Git status, commit, push. Quick check that everyone pushed Day 2 work to GitHub.

00:10 - 00:25



The Streamlit Run-Loop

The mental model: every interaction re-runs your whole script. This is the shift from scripts to apps.

00:25 - 00:55



Live Demo: Build an App Incrementally

Start with `st.title`, add `text_input`, add a button, add balloons, add a conditional. Five steps, one file.

00:55 - 01:10



The st.* Command Cheatsheet

title, write, text_input, button, slider, selectbox, radio, checkbox, columns, sidebar. The 10 commands you'll use 90% of the time.

01:10 - 01:25



Closing Exercise: Unit Converter

Build a length/weight/temperature converter with a selectbox, a number input, and live result.

01:25 - 01:30



Q&A and Day 5 Preview

Open floor, then a quick look at Widgets & Layout.

Yesterday you learned to save your code. Today we make it clickable.

Quick check that your Day 2 work is on GitHub. If it's not, we fix it now — you'll need that code for the group project in Phase 4.

WHAT WE COVERED YESTERDAY DAY 03

- ✓ **Why Git matters** — the two-people-one-file problem, solved
- ✓ **Git vs GitHub** — the tool vs the place (and the email-match gotcha)
- ✓ **The core loop** — status / add / commit / push, typed live
- ✓ **.gitignore** — what NOT to commit (`.venv` , secrets, `__pycache__`)
- ✓ **Merge conflicts** — escalating demos, from easy auto-merge to manual resolution

CHECK YOUR GITHUB NOW VERIFY

```
# Open your repo in a browser: #
github.com/yourname/day3-practice # You should
see: # - clean_contacts.py # - contacts.txt # -
.gitignore # - README.md # - A green contribution
square for yesterday # Verify locally — clone to
a fresh folder: $ git clone
https://github.com/yourname/day3-practice.git
test $ cd test && python clean_contacts.py
('Aarav', 'aarav@example.com') ('Priya',
'priya@example.com') ...
```

⚠ **IF YOUR REPO IS EMPTY OR `clean_contacts.py` FAILS** — raise your hand. We fix it in 5 minutes before we start Streamlit. Today builds on Day 2, not Day 3.

A Python file that opens in a browser. That's the whole pitch.

Normally, building a web app means learning HTML, CSS, JavaScript, and a backend framework. Streamlit skips all of that — you write Python, it renders the UI.

WITHOUT STREAMLIT

THE TRADITIONAL WAY



HTML

For the structure — headings, buttons, inputs



CSS

For the look — colors, fonts, layout, spacing



JavaScript

For the interactivity — what happens when you click



A backend (Flask, Django, Node)

To handle the Python logic on a server



Deployment config

Nginx, Gunicorn, WSGI, ports, domains

Four new languages. A week of setup. Before you write a single line of useful code.

WITH STREAMLIT

THE PYTHON WAY



Python

The language you already know from Days 1–3



streamlit (pip install)

One library, installed on Day 1



app.py

One file. Functions you already know how to write.



streamlit run app.py

One command. A browser tab opens.



Streamlit Community Cloud

Free deployment, one click, on Day 22

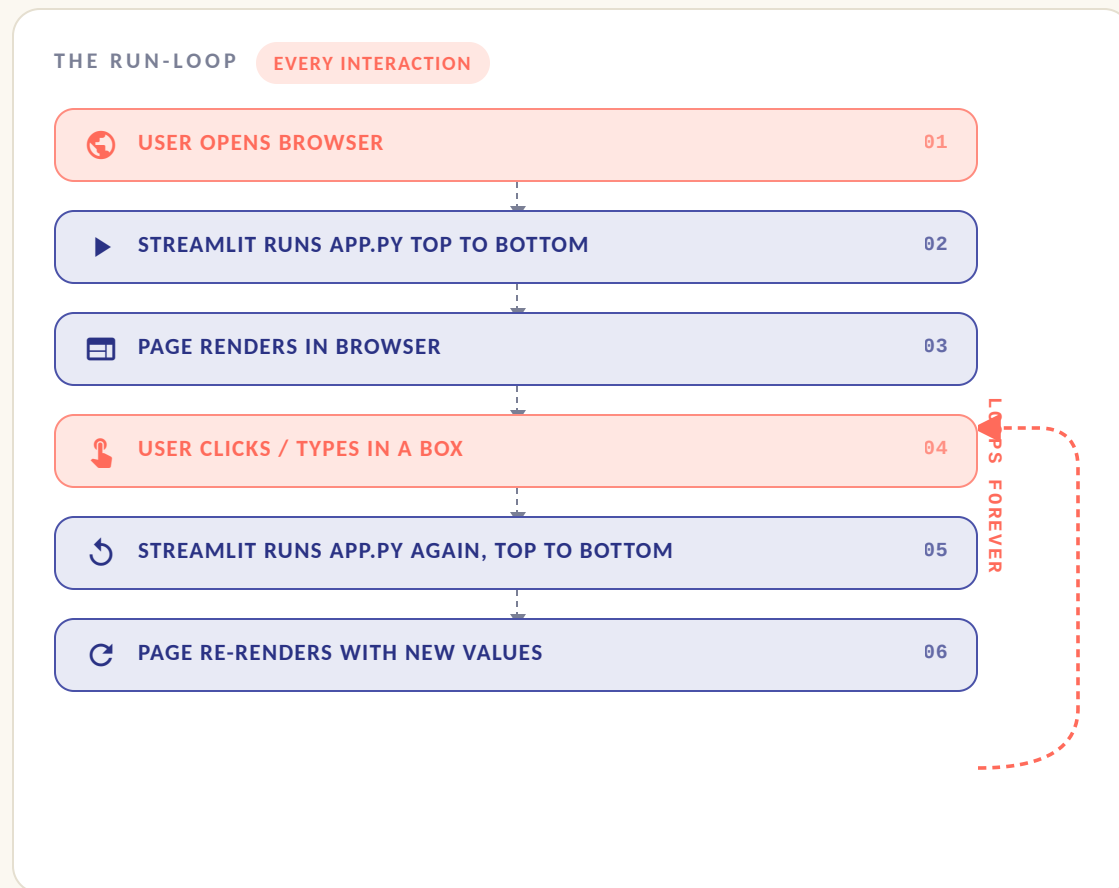
Same language. One file. One command. A working app in 5 minutes.

KEY IDEA

Streamlit is not a web framework. It's a Python library that turns your functions into UI components. `st.title()` is just a function call — it renders a heading. `st.button()` is just a function call — it renders a button and returns **True** or **False**.

Every click re-runs your entire script. Top to bottom.

This is the #1 thing that confuses beginners. A script runs once. A Streamlit app runs every time the user interacts with it. Get this, and everything else clicks.



WHAT THIS MEANS IN PRACTICE

- 1 Variable values update automatically**
When a user types in a `st.text_input`, the variable gets the new value on the next run. You don't write "event handlers" — you just read the variable.
- 2 No state survives between runs (yet)**
Every time the script re-runs, all your regular variables reset. This is by design. We'll cover `st.session_state` on Day 5 for the rare cases you need to remember things.
- 3 Widgets return values, not events**
`st.button("Click me")` returns `True` if the user just clicked it, `False` otherwise. You use it in an `if` statement, like any boolean.
- 4 Order matters**
Code runs top to bottom. If you reference a variable before its widget is defined, you get a `NameError`. Lay out your file like a recipe — inputs first, logic in the middle, output at the bottom.

KEY IDEA A Streamlit app is not a program that waits for events. It's a program that **re-runs itself every time the user does anything**. Once you internalise this, Streamlit feels obvious.

02 / 03 —

Build an app, one line at a time.

Five steps. One file. By the end, you'll have an app that greets you by name, with a button and balloons.

Type along — don't just watch.



Step 1 — A heading on the page.

Open VS Code. Create a new file called `app.py`. Type ONE line. Save. Run `streamlit run app.py` in the terminal. A browser opens. That's it.

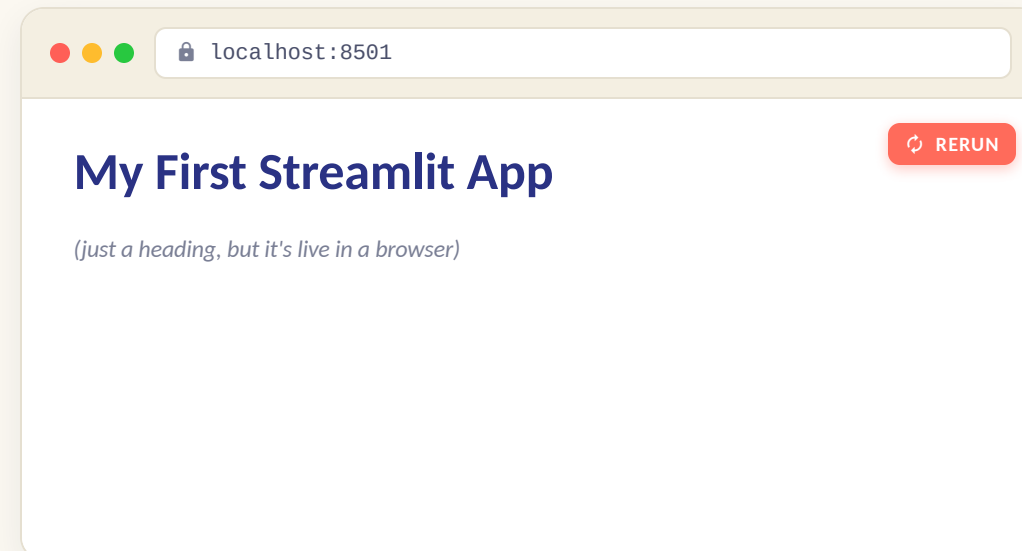
```
app.py PYTHON

# app.py – your first Streamlit app
import streamlit as st

st.title("My First Streamlit App")

# Save the file (Ctrl+S / Cmd+S)
# In the terminal (with .venv active):
# $ streamlit run app.py
#
# A browser tab opens at http://localhost:8501
# You see a page with the heading "My First Streamlit App"
# That's a working app. One import, one function call.
```

BROWSER PREVIEW • WHAT THE STUDENT SEES



▶ One import • one function call • one heading rendered live in the browser.

- 1 `import streamlit as st` — Standard import. We alias as `st` by convention — every Streamlit command starts with `st.`
- 2 `st.title("...")` — A regular Python function call. It renders an H1 heading on the page. The string is the heading text.
- 3 `streamlit run app.py` — The command that starts the Streamlit server. Keep the terminal open — closing

TIP Every time you save `app.py`, Streamlit auto-detects the change and shows a "Rerun" button in the top-right of the browser. Click it (or press `R`) to refresh.

Step 2 — A text box that remembers what you type.

Add ONE line below the title. Save. Click "Rerun" in the browser. Now there's a text box. Type your name — the variable `name` holds whatever you typed.

```
app.py +1 LINE PYTHON

import streamlit as st

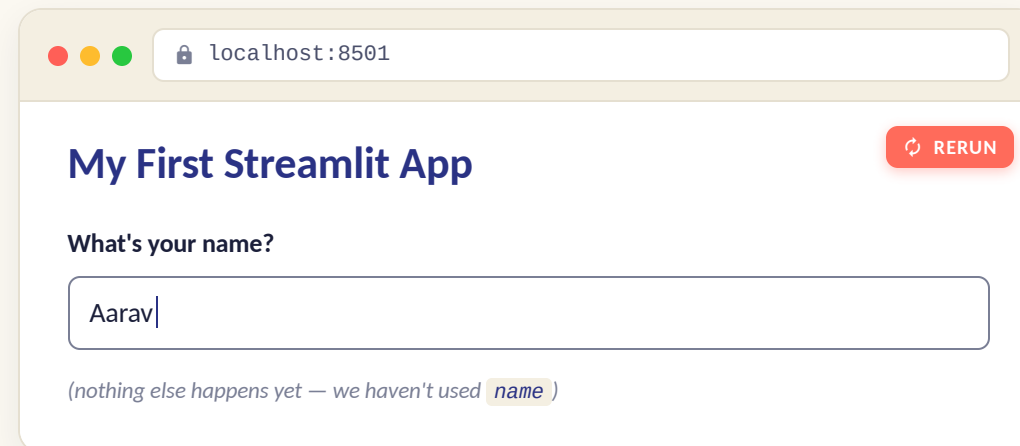
st.title("My First Streamlit App")

# NEW LINE — a text input
name = st.text_input("What's your name?")

# `name` is now a string variable.
# It holds whatever the user typed.
# If empty, it's an empty string "".

# Save. Click "Rerun" in the browser (or press R).
# Type your name in the box.
# Nothing else happens yet — we haven't USED `name`.
# That's Step 3.
```

BROWSER PREVIEW • WHAT THE STUDENT SEES



Each keystroke re-runs the entire script — watch the terminal flicker as you type.

- 1 `st.text_input(...)` — Renders a text box with a label above it. The label string is required — Streamlit uses it for accessibility.
- 2 `name = ...` — The function RETURNS the current value of the text box. Assign it to a variable, just like `input()`.

KEY OBSERVATION Type a character, watch the terminal. You'll see Streamlit re-running your script. That's the **run-loop in action**, visible for the first time.

Step 3 — A button. And finally, we use **name**.

Add a button. When the user clicks it, `st.button` returns True. Use that in an `if` statement — same as any boolean. Now the app responds to a click.

```
app.py +1 LINE PYTHON

import streamlit as st

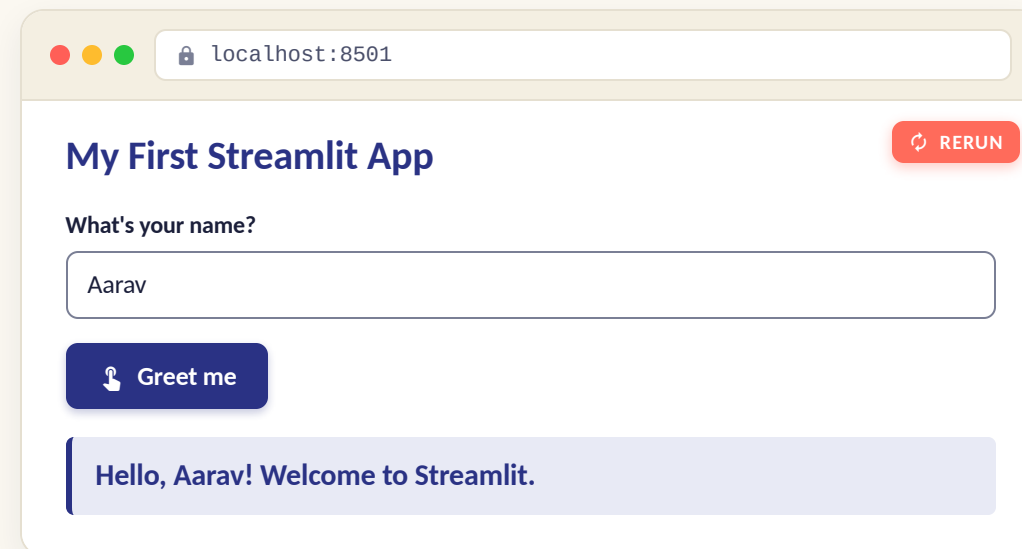
st.title("My First Streamlit App")
name = st.text_input("What's your name?")

# NEW LINE — a button
clicked = st.button("Greet me")

# `clicked` is True if the user just clicked,
# False otherwise.

# Use it in an if statement:
if clicked and name:
```

BROWSER PREVIEW • WHAT THE STUDENT SEES



Click → `clicked=True` → greeting renders. Refresh → button resets.

- 1 `st.button(...)` — Renders a button. Returns True on the run triggered by the click, False on every other run. This is the run-loop in action.
- 2 `clicked and name` — Standard Python boolean logic. We check BOTH that the button was clicked AND that the user typed a name. Without `name`, we'd greet an empty string.
- 3 `st.write(...)` — The Swiss-army knife of Streamlit. Renders almost anything — strings, dataframes, charts, even Matplotlib figures. For now, it prints a string.

KEY IDEA `st.button` does NOT set up an event handler. It returns a boolean. You use it in an `if`. This is the Streamlit mental model — **widgets are functions that return values, not events**.

Step 4 — Balloons. Because every first app deserves a celebration.

One line. When the user clicks the button with a name typed in, balloons animate across the screen. A real, visual reward for a real interaction.

```

app.py +1 LINE PYTHON

import streamlit as st

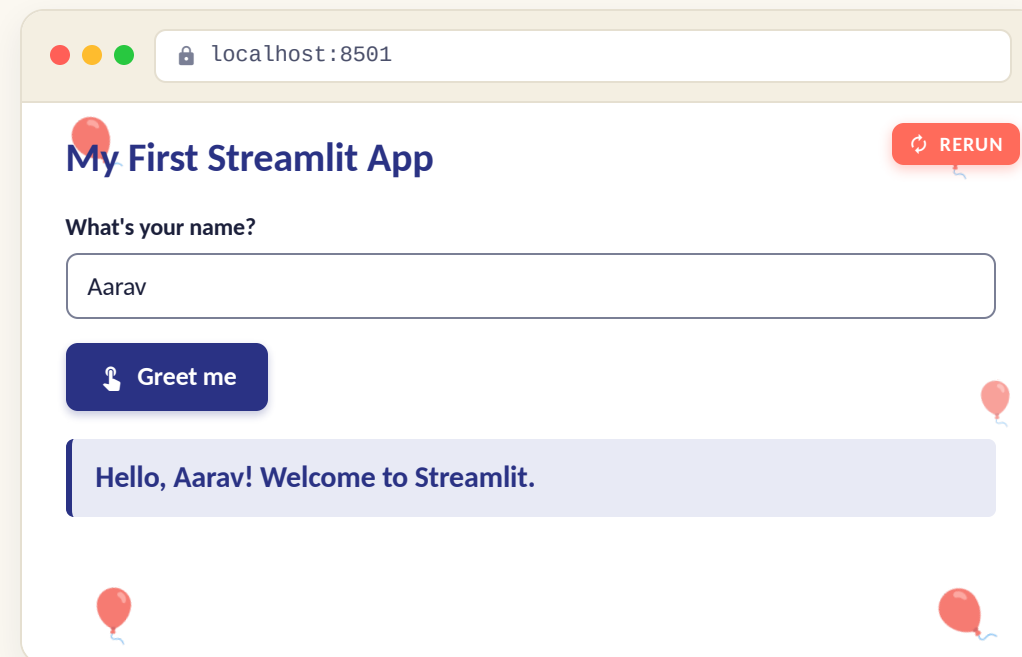
st.title("My First Streamlit App")
name = st.text_input("What's your name?")
clicked = st.button("Greet me")

if clicked and name:
    st.write(f"Hello, {name}! Welcome to Streamlit.")
    # NEW LINE — celebrate!

st.balloons()

# Save. Rerun. Type your name. Click.
# Balloons fly across the screen.
# That's a real interaction — user input,
# a click, a conditional, and a visual reward.
# Five lines of Python. No HTML. No CSS.
# No JavaScript. No backend setup.
    
```

BROWSER PREVIEW • CLICK + NAME = BALLOONS



Balloons rise across the page for ~3 seconds — then settle.

1 **st.balloons()** — A pure fun function. Renders an animation of balloons rising across the screen. No arguments,

WHY THIS MATTERS The balloons aren't the point. The point is that a **5-line Python file** just produced a working, interactive, browser-based app. That's what Streamlit does. That's what the next 20 days build on.

from today is working. The run-loop, widgets as functions, conditionals driving UI — all of it.

Step 5 — What happens when the button ISN'T clicked?

Add an `else` branch. Now the app shows different content depending on state. This is the run-loop in action — the same file renders different pages.

```
import streamlit as st

st.title("My First Streamlit App")
name = st.text_input("What's your name?")
clicked = st.button("Greet me")

if clicked and name:
    st.write(f"Hello, {name}! Welcome to Streamlit.")
    st.balloons()
else:
    # NEW — what shows when NOT clicked

    if name:
        st.write(f"Hi {name}. Click the button above for a surprise.")
    else:
        st.write("Type your name above, then click the button.")
```

<> READING THE NEW CODE

- 1 **The `else` branch** — Standard Python. When `clicked` and `name` is False, the else runs. This is how you show different UI in different states — same file, different output.
- 2 **Nested `if` inside `else`** — We check `name` again inside the else, to give a more helpful message. The pattern: branch on user state, give feedback at every step.
- 3 **Three possible outputs** — The same file now renders three different pages depending on the values of `name` and `clicked`. No router. No page variable. Just conditionals.

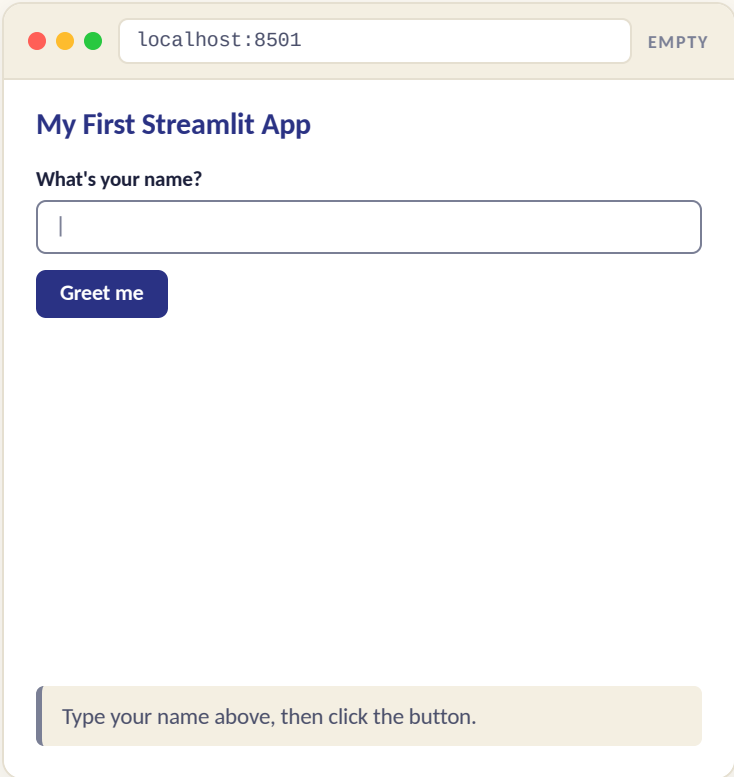
KEY IDEA

There is no “current page” variable. There is no router. The same Python file renders different output based on the values of `name` and `clicked`. **The run-loop handles the rest.** Next slide: the three states visualised.

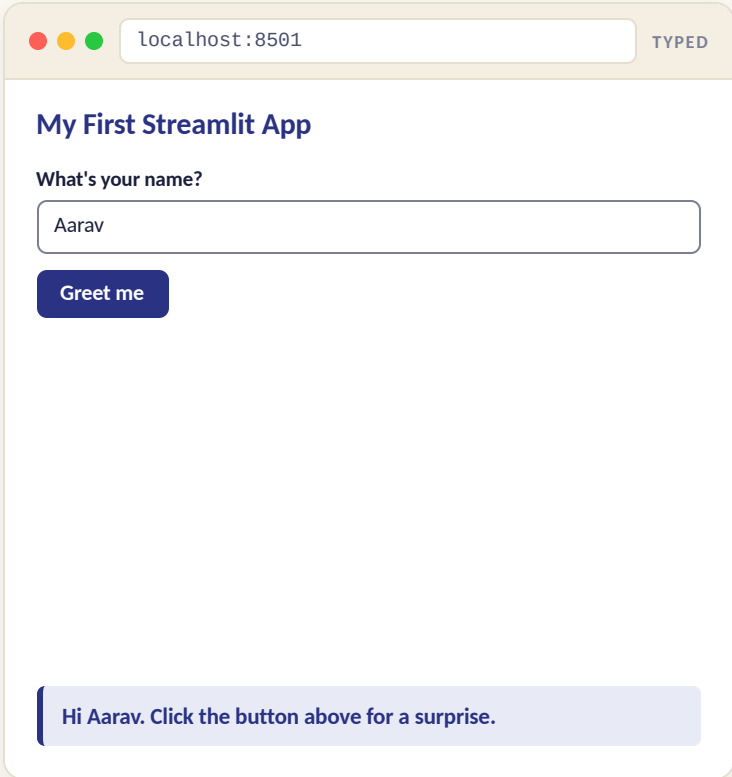
Same file. Three different rendered pages.

The values of `name` and `clicked` decide which page you see. No router. No page variable. Just Python conditionals + the run-loop.

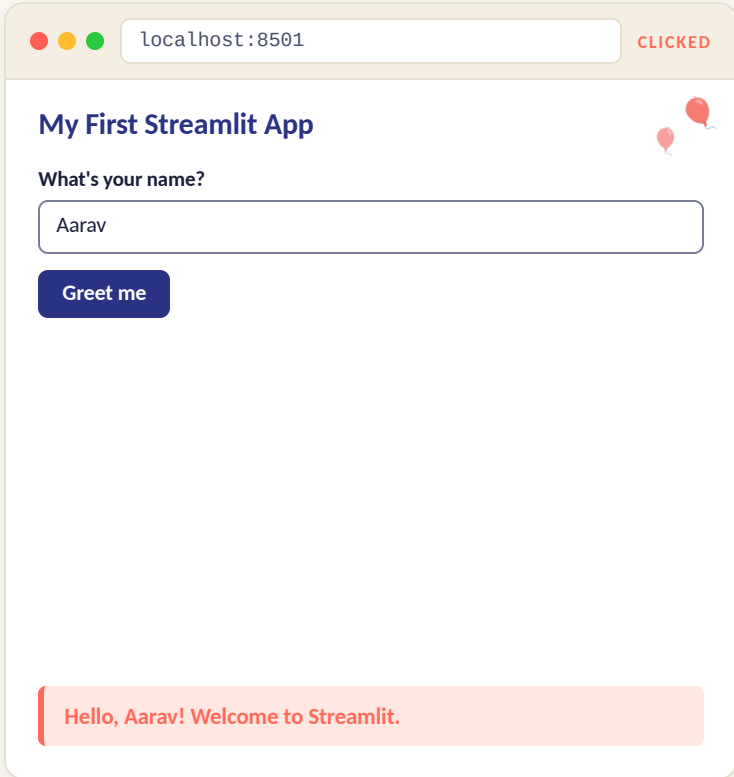
1 No name, no click
name="" · clicked=False



2 Name typed, no click
name="Aarav" · clicked=False



3 Name + click
name="Aarav" · clicked=True



THE PATTERN

Every Streamlit app you build will follow this shape: **read widget values** → **branch on them** → **render different output**. The run-loop re-evaluates the whole file on every interaction. There is no hidden state.

Bookmark this slide. These 10 commands are 90% of Streamlit.

You've seen 4 of these in the live demo. Here are the other 6, with a one-line example each. We'll use all of them by Day 8.

`st.title(text)`

01

Renders an H1 heading. The biggest text on the page.

```
st.title("My App")
```

`st.write(...)`

02

Renders almost anything — strings, dataframes, charts, figures. The Swiss-army knife.

```
st.write(f"Hello, {name}!")
```

`st.text_input(label)`

03

Renders a text box. Returns the typed string.

```
name = st.text_input("Your name")
```

`st.button(label)`

04

Renders a button. Returns True on the click run, False otherwise.

```
if st.button("Click me"): ...
```

`st.slider(label, min, max, default)`

05

Renders a slider. Returns the current number.

```
age = st.slider("Age", 0, 100, 20)
```

`st.selectbox(label, options)`

06

Renders a dropdown. Returns the selected option.

```
color = st.selectbox("Color", ["Red", "Green", "Blue"])
```

`st.radio(label, options)`

07

Renders radio buttons. Returns the selected option.

```
size = st.radio("Size", ["S", "M", "L"])
```

`st.checkbox(label)`

08

Renders a checkbox. Returns True/False.

```
if st.checkbox("Show advanced"): ...
```

`st.columns(n)`

09

Splits the page into n side-by-side columns. Use as a context manager.

```
col1, col2 = st.columns(2)
with col1: st.write("Left")
```

`st.sidebar`

10

A special container for the left sidebar. Access widgets through it.

```
name = st.sidebar.text_input("Your name")
```

PATTERN — Every `st.*` command is just a Python function. It renders something on the page AND returns a value. You call it, you assign the return, you use it. That's the whole API.

03 / 03

Build a unit converter. With a selectbox.

Three categories — length, weight, temperature. A number input. A live result. The pattern you'll reuse in every Streamlit app from here on.

1 BRIEF • 5 MIN

2 BUILD • 10 MIN

3 EXTEND • 5 MIN

Build a length, weight, and temperature converter.

A selectbox to pick the category. A number input for the value. A live result that updates as you type. The pattern: input → logic → output.

TASK

Build a Streamlit app called `converter.py` that lets the user pick a conversion category (length, weight, or temperature), enter a value, and see the converted result live.

REQUIREMENTS

- ✓ Use `st.selectbox` to pick the category: ["Length", "Weight", "Temperature"]
- ✓ Use `st.number_input` for the value to convert
- ✓ Show the result with `st.write` or `st.success`
- ✓ Length: meters ↔ feet (1 m = 3.281 ft)
- ✓ Weight: kg ↔ pounds (1 kg = 2.205 lb)
- ✓ Temperature: Celsius ↔ Fahrenheit ($^{\circ}\text{F} = ^{\circ}\text{C} \times 9/5 + 32$)

BY THE END

Every student has a working converter with at least one category. Two categories is good. All three is excellent.

THE BRIEF

STARTING SCAFFOLD

CONVERTER.PY

```
# converter.py - starting scaffold
import streamlit as st

st.title("Unit Converter")

# 1. Pick the category
category = st.selectbox(
    "What do you want to convert?",
    ["Length", "Weight", "Temperature"]
)

# 2. Get the value
value = st.number_input("Enter a value:", value=0.0)

# 3. Branch on category and compute
if category == "Length":
    # TODO: convert meters ↔ feet
    result = value * 3.281
    unit = "feet"
elif category == "Weight":
    # TODO: convert kg ↔ pounds
    result = value * 2.205
    unit = "pounds"
elif category == "Temperature":
    # TODO: convert Celsius ↔ Fahrenheit
    result = value * 9/5 + 32
```

RUN IT — `streamlit run converter.py`. Try all three categories. The result should update live as you type or change the dropdown.

Finished early? Make it harder. Or read the pitfalls.

Three extension ideas for fast students. Three pitfalls everyone hits. Pick one of each.

EXTENSION IDEAS

FOR FAST STUDENTS

- 1 Add a direction toggle**
Instead of always converting m→ft, add a `st.radio` to let the user pick direction: "m → ft" or "ft → m". Branch your logic on it. Now your app handles 6 conversions instead of 3.
- 2 Format the result nicely**
Use `st.metric` instead of `st.write` for the result. It renders a big number with a label and an optional delta. Looks more like a real dashboard. Try `st.metric(label="Result", value=f"{result:.2f} {unit}")`.
- 3 Add a sidebar with history**
Use `st.sidebar` to add a notes panel where the user can type what they just converted. (Note: this won't persist across reruns yet — that's `st.session_state`, coming on Day 5.)

PITFALLS YOU'LL HIT

EVERYONE HITS THESE

-  **Widget returns None on first render**
Some widgets (like `st.selectbox` with no default) return `None` before the user picks. Always set a default, or check `if value is not None:` before using it.
-  **Forgetting to assign the return value**
Writing `st.button("Click")` without `clicked = ...` means the click is lost. Every widget returns a value — capture it.
-  **Variables don't persist between runs**
If you set `counter = 0` at the top of your file and try to increment it on button click, it resets to 0 every run. This is by design. Use `st.session_state` (Day 5) when you really need persistence.

IF YOU'RE STUCK — Pair with a neighbour. Read the error message out loud (half of debugging is just reading carefully). Raise your hand if both fail. We move at the room's pace.

Today your Python became an app.

Tomorrow we go deeper — widgets that organise, layouts that look intentional, the sidebar pattern. The apps start looking like real products.

OPEN THE FLOOR

Ask Now

- ✓ "Did the run-loop mental model click, or does it still feel weird?"
- ✓ "Which st.* command from the cheatsheet confused you?"
- ✓ "Anything from the unit converter that didn't work?"

BETWEEN SESSIONS

Get Unstuck

- ✓ **Email** • hello@zlaark.com
For Streamlit questions, paste your app.py and the error.
- ✓ **Web** • www.zlaark.com
Program updates and resources.
- ✓ **In-session** • raise a hand
No question is too small. We move at the room's pace.

WHAT'S NEXT • DAYS 5-8

Phase 2 Continues

- ✓ **Day 5 — Widgets & Layout**
Sliders, dropdowns, columns, sidebar.
- ✓ **Days 6-7 — Data & charts**
pandas, line charts, bar charts, dashboards.
- ✓ **Day 8 — Solo challenge**
The gate before group formation on Day 15.